Assignment 5: Web-based Vulnerabilities & Attacks

1. Bypass the Login Page (SQL Injection)

Vulnerability: The login page is vulnerable to SQL Injection because user input in the "password" field is concatenated directly into the database query without sanitization. This allows an attacker to manipulate the query logic.

Approach: We exploited this by injecting a condition that always evaluates to true (OR 1=1). This forces the database to return a valid record even though the password is incorrect, allowing us to log in as the default user.

Payload Used (Remote Server): Due to a security filter (WAF) blocking spaces on the remote server, we replaced standard spaces with SQL comment blocks /**/.

- **Username:** user
- **Password:** '/**/OR/**/1=1--

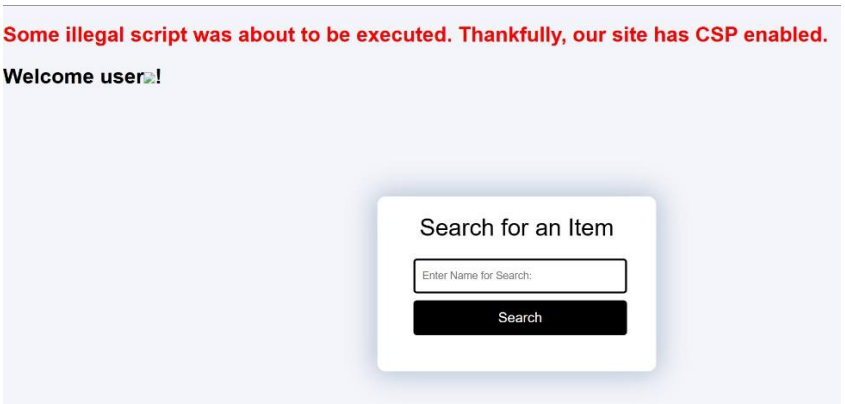
2. Exploit DOM-XSS

Vulnerability: A DOM-based Cross-Site Scripting (XSS) vulnerability exists in the greet.js script. The application reads the URL hash (everything after #) and writes it directly into the HTML of the page using document.write() without sanitization.

Approach: We bypassed the browser's XSS auditor by using an image tag with an error handler (<img onerror=...>) instead of a standard <script> tag. When the page loads, the browser tries to load the non-existent image "x", fails, and immediately executes our malicious JavaScript alert.

Payload Used:

- **URL:** http://139.91.71.5:11337/dashboard#user<img src=x onerror=alert(1)>



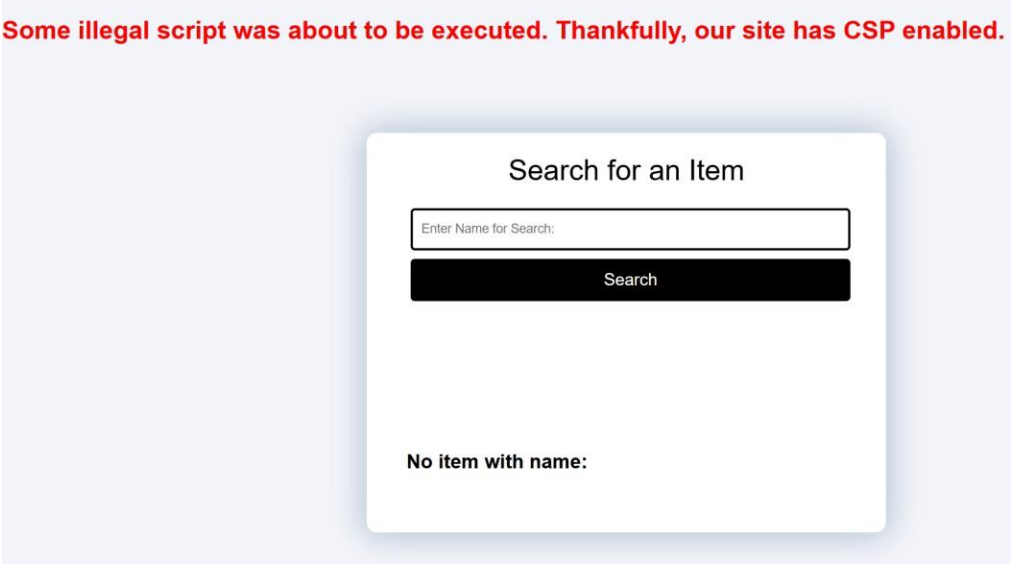
3. Exploit Reflected XSS

Vulnerability: The search functionality in the dashboard is vulnerable to Reflected XSS. When a user searches for an item that does not exist, the server "reflects" the input back to the user in a flash message without escaping special HTML characters.

Approach: We injected an SVG element with an onload event handler into the search bar. When the server returned the page with the "Item not found" message, our malicious tag was rendered, triggering the JavaScript execution.

Payload Used:

- **Form Field:** Search Box
- **Payload:** <svg onload=alert(1)>



4. SQL Injection in Search (Data Extraction)

Vulnerability: The search endpoint (/search) is vulnerable to SQL Injection. By using a UNION operator, we can append results from other database tables (specifically the users table) to the legitimate item search results.

Approach: We encountered two obstacles on the remote server:

1. **Space Filter:** We bypassed this using /**/ comments.
2. **Column Mismatch:** The query required exactly 3 columns to match the main query structure.

We constructed a payload that selected the username and password columns from the users table and displayed them as if they were search results.

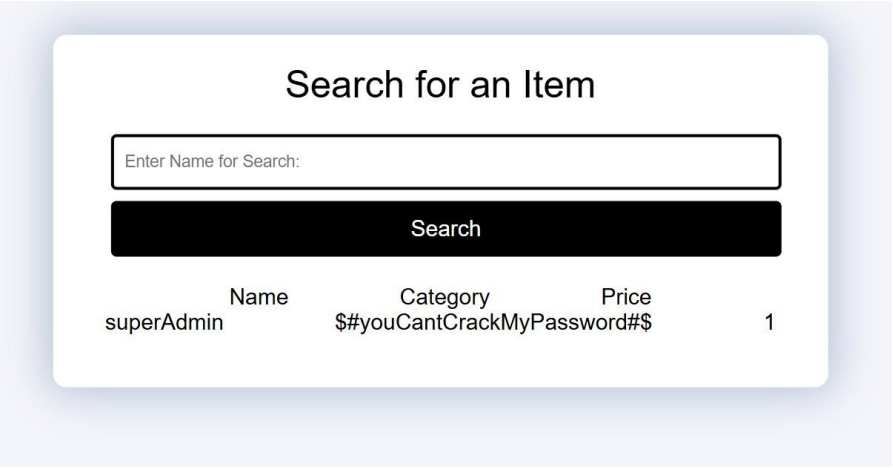
Payload Used:

SQL

'/**/UNION/**/SELECT/**/username,password,1/**/FROM/**/users/**/--/**/

Result: The injection successfully revealed the administrator credentials:

- **Username:** superAdmin
- **Password:** $#youCantCrackMyPassword#$



5. Login as Admin

Approach: Using the credentials retrieved in the previous step, we navigated to the Admin portal and authenticated successfully.

- **URL:** /admin
- **Username:** superAdmin
- **Password:** $#youCantCrackMyPassword#$

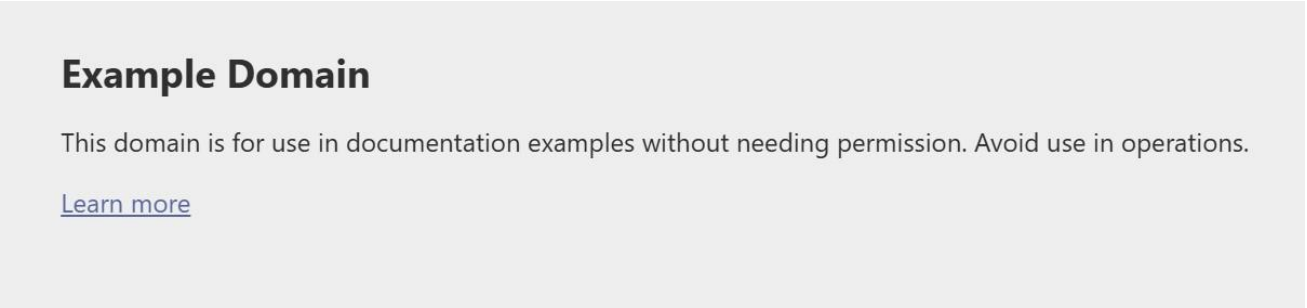
6. Exploit Open Redirect

Vulnerability: The application includes a utility endpoint (/go) that takes a to parameter and redirects the user to that URL. The application fails to validate whether the destination is internal or external, allowing for Open Redirection attacks.

Approach: We manipulated the URL parameter to redirect the user to an arbitrary external domain (example.com) as required by the assignment.

Payload Used:

- **URL:** http://139.91.71.5:11337/go?to=http://example.com



7. Exploit Local File Inclusion (LFI) & The Secret Flag

Vulnerability: The Admin Dashboard's file viewer (?show=) is vulnerable to Local File Inclusion (LFI). While the application attempts to secure the input by blocking dots (.) to prevent directory traversal (../), it fails to restrict access to valid subdirectories within the application root.

Approach: Through source code analysis (white-box testing), we identified that:

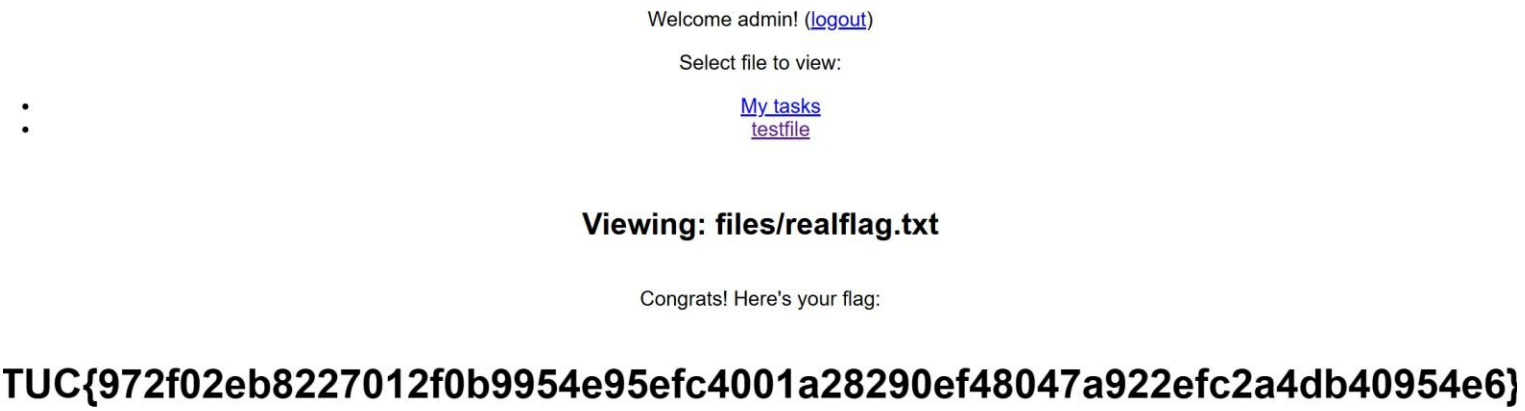
1. The server automatically appends .txt to the input.
2. The "no dot" filter prevents us from going *up* the directory tree, but not *down*.
3. We discovered a file named realflag.txt located inside the files/ directory by listing the directory contents.

We exploited the vulnerability by providing the relative path to the flag file without the extension.

Payload Used:

- **URL Parameter:** ?show=files/realflag
- **Full URL:** http://139.91.71.5:11337/admin?show=files/realflag

The Secret Flag:

Plaintext

TUC{972f02eb8227012f0b9954e95efc4001a28290ef48047a922efc2a4db40954e6}